

MAS Coding Lab

Agent Memory & Knowledge Graph

Implementing Multi-Agent Systems with Neo4j Knowledge Graph Memory

- CrewAI
- Google ADK
- LangGraph
- Neo4j
- GraphRAG

Agentic AI Coding Fitness • Week 9
agentic-coding-fitness.altotech.ai • AltoTech Global

Reference: neo4j.com/labs/genai-ecosystem/llm-graph-builder/

Prerequisites: mas_environment_setup.pdf • mas_coding_exercises.pdf • Docker + Neo4j

Introduction

From Design Thinking to Working Code + Knowledge Graphs

This lab guide bridges your MAS Design Thinking Workshop scenarios with hands-on Python code. Each of the 10 business scenarios you designed on paper is now implemented as a real Multi-Agent System, enhanced with agent memory backed by a Neo4j knowledge graph.

What You Will Build

Component	Technology	Purpose
Multi-Agent Orchestration	CrewAI / Google ADK / LangGraph	Coordinate specialized agents
Agent Working Memory	Python dict / session.state	In-context short-term state
Episodic Memory	SQLite / Redis log	History of past agent actions
Semantic Memory	ChromaDB / vector embeddings	Similarity search on knowledge
Knowledge Graph Memory	Neo4j + LangChain	Structured entity-relationship recall
GraphRAG Pipeline	LangChain GraphCypherQChain	Natural language → Cypher → context

Lab Structure

Each lab contains three parts:

Part A — MAS Core	Multi-agent implementation of the scenario using CrewAI, Google ADK, or LangGraph. Agents, tools, tasks, and orchestration.
Part B — KG Memory Integration	Neo4j schema for the scenario. Python code to store events and recall context using Cypher queries.
Part C — Challenges	3 exercises to extend the system: new agents, new graph relationships, GraphRAG queries.

Key Insight:

The knowledge graph is not just storage — it is the agent's long-term structured memory. Agents can traverse relationships, find patterns, and answer "Why did we make this decision 3 days ago?" in a way that flat databases cannot.

Chapter 1: Agent Memory Architecture

Four memory types — from ephemeral context to persistent knowledge graph

Intelligent agents need memory to act coherently across time. Woolridge (2002, Ch.2) defines an agent as a system that perceives its environment and acts upon it to achieve goals — but without memory, each perception is isolated. The four-level memory hierarchy below mirrors human cognition.

■ Woolridge — Introduction to MAS (Ch. 2, 4)

A fully rational agent selects actions that maximise expected utility given its beliefs (BDI model). Beliefs are the agent's internal model of the world — precisely what memory provides. Without persistent memory, an agent cannot form beliefs that outlast a single perception-action cycle.

The Four Memory Layers

1	Working Memory Variables, session.state, prompt context window <code>session.state["last_reading"] = 24.5</code>	In-Context (Ephemeral) Speed: Fast (microseconds) Durability: Vanishes at session end
2	Episodic Memory SQLite / Redis list of timestamped events <code>log_event(agent, "sensor_read", data)</code>	Short-Term Log Speed: Fast (milliseconds) Durability: Hours to days
3	Semantic Memory ChromaDB / Pinecone — similarity search <code>collection.query(query_texts=["hot room"])</code>	Vector Embeddings Speed: Medium (100ms) Durability: Permanent
4	Knowledge Graph Neo4j — entities, edges, Cypher traversal <code>MATCH (r:Room)-[:HAS]->(d:Device) RETURN r,d</code>	Structured Relationships Speed: Medium (50–200ms) Durability: Permanent

Why Knowledge Graphs Win for Agent Memory

While vectors find similar content, knowledge graphs find related entities. An agent storing "Room 301 temperature spike caused HVAC override" as a graph edge can later traverse: Which rooms had overrides triggered by the same HVAC unit? — a multi-hop query impossible in a vector store.

Capability	Vector DB	Knowledge Graph
Find similar past events	✓ Excellent	○ Possible via embeddings
Multi-hop relationship queries	× Not supported	✓ Native (Cypher)
Causal chain reconstruction	×	✓ MATCH path queries
Entity deduplication	○ Approximate	✓ MERGE node patterns
Structured schema enforcement	○ Partial	✓ Constraints + indexes

Capability	Vector DB	Knowledge Graph
Natural language → query (GraphRAG)	✓ Semantic search	✓ Text2Cypher

Key Insight: Use both: vectors for fuzzy recall ("find similar scenarios"), knowledge graphs for precise structured reasoning ("find all agents that accessed customer X in the last 24 hours"). The Neo4j LLM Graph Builder creates both automatically.

Chapter 2: Neo4j Knowledge Graph Setup

Docker install · Python driver · LangChain integration · LLM Graph Builder

■ Neo4j LLM Graph Builder

An online app + Python backend that converts unstructured text (PDFs, docs, web pages, YouTube) into a Neo4j knowledge graph. It uses LangChain's `llm-graph-transformer` under the hood and provides a React frontend on Google Cloud Run or Docker locally. Reference: neo4j.com/labs/genai-ecosystem/llm-graph-builder/

Step 1 — Run Neo4j with Docker

■ docker-compose.yml

```
# docker-compose.yml (save and run: docker-compose up -d)
version: "3.8"
services:
  neo4j:
    image: neo4j:5.18-community
    ports:
      - "7474:7474" # Browser UI
      - "7687:7687" # Bolt protocol
    environment:
      NEO4J_AUTH: neo4j/mas_memory_2024
      NEO4J_PLUGINS: '["apoc", "graph-data-science"]'
    volumes:
      - neo4j_data:/data
    volumes:
      neo4j_data:
```

After startup, open `http://localhost:7474` and login with `neo4j / mas_memory_2024`. The Bolt endpoint for Python is `bolt://localhost:7687`.

Step 2 — Python Driver

■ neo4j_connect.py

```
# Install: pip install neo4j langchain-neo4j
from neo4j import GraphDatabase

URI = "bolt://localhost:7687"
AUTH = ("neo4j", "mas_memory_2024")

with GraphDatabase.driver(URI, auth=AUTH) as driver:
    driver.verify_connectivity()
    print("Connected to Neo4j")

    with driver.session() as session:
        # Create a test node
        session.run(
            "CREATE (a:Agent {name: $name, created: datetime()})",
            name="SensorAgent"
        )
        # Verify
        result = session.run(
            "MATCH (a:Agent) RETURN a.name AS name"
        )
        for r in result:
            print(r["name"])
```

Step 3 — LangChain Neo4j Integration

■ langchain_neo4j.py

```
# pip install langchain langchain-community langchain-openai
from langchain_community.graphs import Neo4jGraph
from langchain.chains import GraphCypherQAChain
from langchain_openai import ChatOpenAI # or z.ai endpoint

graph = Neo4jGraph(
    url="bolt://localhost:7687",
    username="neo4j",
    password="mas_memory_2024"
)

# Refresh schema (auto-detects nodes/relationships)
graph.refresh_schema()
print(graph.schema)

# Natural language → Cypher → Answer
llm = ChatOpenAI(
    base_url="https://api.z.ai/api/paas/v4/",
    model="glm-4.5-air",
    api_key="YOUR_Z_AI_KEY"
)
chain = GraphCypherQAChain.from_llm(llm, graph=graph, verbose=True)
answer = chain.invoke(
    {"query": "Which rooms had temperature spikes today?"}
)
print(answer["result"])
```

Step 4 — LLM Graph Builder (Auto-Extract)

■ llm_graph_builder.py — Auto-extract entities from text

```
# pip install langchain-experimental
from langchain_experimental.graph_transformers import LLMGraphTransformer
from langchain_community.graphs import Neo4jGraph
from langchain_openai import ChatOpenAI
from langchain_core.documents import Document

llm = ChatOpenAI(
    base_url="https://api.z.ai/api/paas/v4/",
    model="glm-4.5-air",
    api_key="YOUR_Z_AI_KEY"
)

graph = Neo4jGraph(
    url="bolt://localhost:7687",
    username="neo4j",
    password="mas_memory_2024"
)

transformer = LLMGraphTransformer(llm=llm)

# Feed any text document into the knowledge graph
docs = [Document(page_content="Room 301 HVAC unit failed at 14:23. "
    "Maintenance agent dispatched. "
    "Energy consumption increased by 40%."
)]

graph_docs = transformer.convert_to_graph_documents(docs)
```

```
graph.add_graph_documents(graph_docs, include_source=True)
# Now Room, HVAC, MaintenanceAgent nodes exist in Neo4j!
```

■ Graph structure created by LLM Graph Builder

Document node → Chunk nodes (with embeddings + SIMILAR kNN edges) → Entity nodes (Room, HVAC, Agent) + Relationship edges. This mirrors the architecture at neo4j.com/labs/genai-ecosystem/llm-graph-builder/

Chapter 3: The AgentMemory Base Class

Reusable pattern — every lab extends this class

All 10 labs extend a common AgentMemory base class. It abstracts the Neo4j driver, provides store/recall/query methods, and integrates with LangChain for GraphRAG. Copy this to agent_memory.py in every project.

■ agent_memory.py (Part 1 — Core class)

```
"""agent_memory.py — Reusable AgentMemory for all MAS labs."""
```

```
import json
from datetime import datetime
from neo4j import GraphDatabase
from langchain_community.graphs import Neo4jGraph
from langchain.chains import GraphCypherQAChain
```

```
URI = "bolt://localhost:7687"
AUTH = ("neo4j", "mas_memory_2024")
```

```
class AgentMemory:
    """Knowledge-graph-backed memory for any MAS agent."""
```

```
def __init__(self, agent_id: str,
             uri=URI, auth=AUTH):
    self.agent_id = agent_id
    self.driver = GraphDatabase.driver(uri, auth=auth)
    self._ensure_agent_node()
```

```
# — Internal —————
def _ensure_agent_node(self):
    with self.driver.session() as s:
        s.run(
            "MERGE (a:Agent {id: $id}) "
            "ON CREATE SET a.created = datetime()",
            id=self.agent_id
        )
```

```
# — Store —————
def store_event(self, event_type: str,
               data: dict, entities: list = None):
    """
    Write an event node linked to this agent.
    entities: list of (label, name) tuples for
    related domain nodes.
    """
    with self.driver.session() as s:
        event_id = (f"{self.agent_id}:"
                    f"{datetime.now().isoformat()}")
        s.run(
            "MATCH (a:Agent {id: $aid}) "
            "CREATE (e:Event {"
            " id: $eid, type: $etype, "
            " data: $data, ts: datetime()}) "
            "CREATE (a)-[:PERFORMED]->(e)",
            aid=self.agent_id,
            eid=event_id,
            etype=event_type,
            data=json.dumps(data)
        )
    if entities:
```

```

for label, name in entities:
    s.run(
        f"MERGE (n:{label} {{name: $name}}) "
        f"WITH n "
        f"MATCH (e:Event {{id: $eid}}) "
        f"MERGE (e)-[:INVOLVES]->(n)",
        name=name, eid=event_id
    )
    return event_id

# — Recall —————
def recall_recent(self, event_type: str = None,
    limit: int = 5) -> list:
    """Return most recent events for this agent."""
    with self.driver.session() as s:
        if event_type:
            result = s.run(
                "MATCH (a:Agent {id: $aid})"
                "-[:PERFORMED]->(e:Event) "
                "WHERE e.type = $etype "
                "RETURN e ORDER BY e.ts DESC "
                "LIMIT $lim",
                aid=self.agent_id,
                etype=event_type, lim=limit
            )
        else:
            result = s.run(
                "MATCH (a:Agent {id: $aid})"
                "-[:PERFORMED]->(e:Event) "
                "RETURN e ORDER BY e.ts DESC "
                "LIMIT $lim",
                aid=self.agent_id, lim=limit
            )
        return [dict(r["e"]) for r in result]

def recall_related(self, entity_label: str,
    entity_name: str) -> list:
    """Find all events involving a specific entity."""
    with self.driver.session() as s:
        result = s.run(
            f"MATCH (e:Event)-[:INVOLVES]->"
            f"(n:{entity_label} {{name: $name}}) "
            f"RETURN e ORDER BY e.ts DESC LIMIT 10",
            name=entity_name
        )
        return [dict(r["e"]) for r in result]

def cypher(self, query: str, **params) -> list:
    """Run raw Cypher and return dicts."""
    with self.driver.session() as s:
        return [dict(r) for r in s.run(query, **params)]

def close(self):
    self.driver.close()

```

■ agent_memory.py (Part 2 — GraphRAG + usage)

```

# — GraphRAG —————
def build_graphrag_chain(self, llm):
    """
    Return a LangChain chain that converts natural
    language questions into Cypher and answers them.
    """

```

```

graph = Neo4jGraph(
    url=self.driver._pool._address.netloc
    if hasattr(self.driver, "_pool") else URI,
    username=AUTH[0],
    password=AUTH[1]
)
graph.refresh_schema()
return GraphCypherQChain.from_llm(
    llm, graph=graph, verbose=False
)

# — Usage example —————
if __name__ == "__main__":
    mem = AgentMemory("SensorAgent_01")

    # Store an event with related entities
    mem.store_event(
        event_type="temperature_alert",
        data={"room": "301", "temp": 29.5, "threshold": 26},
        entities=[("Room", "Room_301"), ("Alert", "HIGH_TEMP")]
    )

    # Recall recent events
    recent = mem.recall_recent(event_type="temperature_alert")
    print("Recent alerts:", recent)

    # Recall by entity
    related = mem.recall_related("Room", "Room_301")
    print("Room 301 history:", related)

    mem.close()

```

Key Insight:

The `store_event()` method creates Agent→Event→Entity chains in the graph. Over time, Neo4j builds a rich causal memory: "Agent X performed action Y on entity Z at time T." This is what makes agent decisions explainable and auditable.

Scenario Brief

A 200-room hotel wants to cut energy costs by 30%. Occupancy sensors, thermostats, and HVAC units generate data every 60 seconds. A MAS must detect anomalies, optimise HVAC schedules, notify maintenance, and learn from past incidents to prevent recurrence — all without guest disruption.

Agent Roles

Agent Role	Goal	Key Tool / Action
SensorMonitorAgent	Read IoT data, detect anomalies	MQTT subscriber / REST poll
EnergyOptimizerAgent	Compute optimal HVAC schedule	Optimisation solver + room occupancy
MaintenanceAgent	Dispatch work orders, track repairs	CMMS API (e.g. Maximo)
MemoryAgent	Store events to Neo4j KG, answer queries	AgentMemory class + GraphRAG

Knowledge Graph Schema

■ Cypher schema (run once to set up)

```
// Hotel KG Schema
CREATE CONSTRAINT IF NOT EXISTS FOR (r:Room)
  REQUIRE r.id IS UNIQUE;
CREATE CONSTRAINT IF NOT EXISTS FOR (d:Device)
  REQUIRE d.id IS UNIQUE;

// Domain nodes
MERGE (:Room {id:"R301", floor:3, type:"standard"})
MERGE (:Device {id:"HVAC_301", type:"HVAC"})
MERGE (:AlertType {name:"HIGH_TEMP"})

// Relationships
MATCH (r:Room {id:"R301"}), (d:Device {id:"HVAC_301"})
MERGE (r)-[:HAS_DEVICE]->(d);

// Example: an energy event
CREATE (e:EnergyEvent {
  ts: datetime(), room: "R301",
  kw: 3.2, occupancy: true,
  action: "HVAC_REDUCED"
})-[:AFFECTS]->(:Room {id:"R301"});
```

Part A — MAS Core Implementation

■ lab1_hotel_mas.py — CrewAI multi-agent

```
# lab1_hotel_mas.py
from crewai import Agent, Task, Crew, Process
from crewai_tools import BaseTool
from typing import Any

# --- Tools ---
class SensorTool(BaseTool):
    name: str = "read_sensor"
    description: str = "Read room temperature & occupancy"
    def _run(self, room_id: str) -> dict:
        # Replace with real MQTT/REST call
```

```

import random
return {
    "room_id": room_id,
    "temp_c": round(20 + random.random() * 12, 1),
    "occupancy": random.choice([True, False]),
    "kwh": round(1.5 + random.random() * 3, 2)
}

class HVACTool(BaseTool):
    name: str = "set_hvac"
    description: str = "Set HVAC setpoint for a room"
    def _run(self, room_id: str, setpoint: float) -> str:
        print(f" [HVAC] Room {room_id} → {setpoint}°C")
        return f"HVAC_SET:{room_id}:{setpoint}"

# --- Agents ---
sensor_agent = Agent(
    role="Sensor Monitor",
    goal="Detect temperature anomalies across all rooms",
    backstory="IoT specialist monitoring 200 hotel rooms",
    tools=[SensorTool()],
    verbose=False
)
optimizer = Agent(
    role="Energy Optimizer",
    goal="Minimize energy while keeping guests comfortable",
    backstory="Energy engineer with ML optimization skills",
    tools=[HVACTool()],
    verbose=False
)

# --- Tasks ---
scan = Task(
    description="Scan rooms R301..R310. Flag any room "
    "above 27°C or consuming > 4 kWh.",
    expected_output="List of anomalous rooms with readings",
    agent=sensor_agent
)
optimize = Task(
    description="For each flagged room, set HVAC to "
    "reduce temp by 2°C. Return actions taken.",
    expected_output="Dict of room_id→new_setpoint",
    agent=optimizer
)

# --- Crew ---
crew = Crew(
    agents=[sensor_agent, optimizer],
    tasks=[scan, optimize],
    process=Process.sequential,
    verbose=False
)

if __name__ == "__main__":
    result = crew.kickoff()
    print(result)

```

Part B — Neo4j Knowledge Graph Memory

■ lab1_hotel_memory.py — Neo4j KG memory

```

# lab1_hotel_memory.py - Neo4j KG memory integration
from agent_memory import AgentMemory

```

```

mem = AgentMemory("HotelEnergyMAS")

def store_reading(room_id, reading: dict):
    """Store sensor reading as event in KG."""
    mem.store_event(
        event_type="sensor_reading",
        data=reading,
        entities=[
            ("Room", room_id),
            ("Device", f"HVAC_{room_id[1:]}")
        ]
    )

def store_action(room_id, action, setpoint):
    """Store HVAC action in KG."""
    mem.store_event(
        event_type="hvac_action",
        data={"action": action, "setpoint": setpoint},
        entities=[("Room", room_id)]
    )

# --- Recall patterns ---
def rooms_with_recent_alerts(hours=24):
    return mem.cypher("""
MATCH (e:Event {type: "sensor_reading"})
-[:INVOLVES]->(r:Room)
WHERE e.ts > datetime() - duration({hours: $h})
AND apoc.convert.fromJsonMap(e.data).temp_c > 27
RETURN r.name AS room,
count(e) AS alert_count
ORDER BY alert_count DESC
""", h=hours)

def most_active_hvac_rooms(limit=5):
    return mem.cypher("""
MATCH (e:Event {type: "hvac_action"})
-[:INVOLVES]->(r:Room)
RETURN r.name AS room,
count(e) AS actions
ORDER BY actions DESC LIMIT $lim
""", lim=limit)

# --- Run ---
if __name__ == "__main__":
    store_reading("R301",
{"temp_c": 28.3, "occupancy": True, "kwh": 4.1})
    store_action("R301", "HVAC_REDUCED", 22.0)
    print("Rooms with alerts:",
rooms_with_recent_alerts())
    mem.close()

```

Key Insight: The KG stores the history of every HVAC action linked to rooms. A week later, an agent can ask: "Which rooms have had more than 3 temperature overrides this month?" — a query that drives predictive maintenance scheduling.

Part C — Challenges

C1.1

Add a MaintenanceAgent that reads recall_related("Room", "R301") and auto-generates a work order if a room has had >3 HVAC actions in 24 hours.

C1.2	Use the LLM Graph Builder (Chapter 2 Step 4) to ingest your hotel floor plan PDF and auto-extract Room→Device relationships.
C1.3	Build a GraphRAG chain that answers: "Which floor had the highest energy use last week?" using GraphCypherQAChain.

Scenario Brief

An online retailer processes 10,000 orders/day across 5 warehouses. When inventory is low, agents must re-route orders, trigger restocking, negotiate delivery slots, and communicate ETAs to customers — all within 30 seconds per order. The KG tracks supplier reliability and shipping history to improve routing decisions.

Agent Roles

Agent Role	Goal	Key Tool / Action
InventoryAgent	Monitor stock levels, trigger restock alerts	Warehouse ERP API
OrderRoutingAgent	Assign orders to optimal warehouse	Distance + stock optimisation
SupplierAgent	Negotiate restock quantities and lead times	Supplier REST API / email
NotificationAgent	Send customer ETA updates	SMS/email gateway

Knowledge Graph Schema

■ Cypher schema (run once to set up)

```
// E-Commerce Fulfillment KG
CREATE CONSTRAINT IF NOT EXISTS FOR (o:Order)
  REQUIRE o.id IS UNIQUE;
CREATE CONSTRAINT IF NOT EXISTS FOR (p:Product)
  REQUIRE p.sku IS UNIQUE;
CREATE CONSTRAINT IF NOT EXISTS FOR (w:Warehouse)
  REQUIRE w.id IS UNIQUE;

MERGE (:Warehouse {id:"WH_BKK", city:"Bangkok",
  capacity:50000})
MERGE (:Warehouse {id:"WH_SGP", city:"Singapore",
  capacity:30000})
MERGE (:Product {sku:"SKU_001", name:"Smart Sensor",
  reorder_pt:100})

// Relationships
MATCH (w:Warehouse {id:"WH_BKK"}),
  (p:Product {sku:"SKU_001"})
MERGE (w)-[:STOCKS {qty:450}]->(p);
```

Part A — MAS Core Implementation

■ lab2_ecommerce_mas.py — LangGraph stateful workflow

```
# lab2_ecommerce_mas.py - LangGraph workflow
from typing import TypedDict, List
from langgraph.graph import StateGraph, END

class FulfillmentState(TypedDict):
    order_id: str
    sku: str
    qty: int
    assigned_warehouse: str
    status: str
    actions: List[str]

# --- Node functions ---
```

```

def check_inventory(state: FulfillmentState):
    """Agent: check stock across warehouses."""
    # Replace with real ERP call
    stock = {"WH_BKK": 450, "WH_SGP": 80}
    best_wh = max(stock, key=stock.get)
    if stock[best_wh] >= state["qty"]:
        return {**state,
                "assigned_warehouse": best_wh,
                "status": "IN_STOCK",
                "actions": state["actions"]
                + [f"stock_ok:{best_wh}"]}
    return {**state, "status": "LOW_STOCK",
            "actions": state["actions"] + ["low_stock"]}

def route_order(state: FulfillmentState):
    """Agent: create shipment at assigned warehouse."""
    wh = state["assigned_warehouse"]
    return {**state,
            "status": "ROUTED",
            "actions": state["actions"]
            + [f"routed_to:{wh}"]}

def trigger_restock(state: FulfillmentState):
    """Agent: send restock request to supplier."""
    return {**state,
            "status": "RESTOCK_REQUESTED",
            "actions": state["actions"]
            + ["restock_req:SKU_001"]}

def notify_customer(state: FulfillmentState):
    """Agent: send ETA notification."""
    eta = "2 business days"
    print(f" [SMS] Order {state['order_id']} → ETA {eta}")
    return {**state, "status": "NOTIFIED"}

# --- Graph ---
graph = StateGraph(FulfillmentState)
graph.add_node("inventory", check_inventory)
graph.add_node("route", route_order)
graph.add_node("restock", trigger_restock)
graph.add_node("notify", notify_customer)

graph.set_entry_point("inventory")
graph.add_conditional_edges("inventory", lambda s: s["status"],
                             {"IN_STOCK": "route", "LOW_STOCK": "restock"})
graph.add_edge("route", "notify")
graph.add_edge("restock", "notify")
graph.add_edge("notify", END)
app = graph.compile()

if __name__ == "__main__":
    result = app.invoke({
        "order_id": "ORD_20240519_001",
        "sku": "SKU_001", "qty": 5,
        "assigned_warehouse": "",
        "status": "PENDING", "actions": []
    })
    print(result)

```

Part B — Neo4j Knowledge Graph Memory

■ lab2_ecommerce_memory.py — Order routing KG

```

# lab2_ecommerce_memory.py
from agent_memory import AgentMemory

mem = AgentMemory("FulfillmentMAS")

def store_order_routed(order_id, sku, warehouse):
    mem.store_event(
        event_type="order_routed",
        data={"order": order_id, "sku": sku,
              "warehouse": warehouse},
        entities=[
            ("Order", order_id),
            ("Product", sku),
            ("Warehouse", warehouse)
        ]
    )

def store_restock(sku, supplier, qty):
    mem.store_event(
        event_type="restock_triggered",
        data={"sku": sku, "supplier": supplier,
              "qty": qty},
        entities=[("Product", sku), ("Supplier", supplier)]
    )

# --- KG analytics ---
def warehouse_load_today():
    """How many orders each warehouse handled today."""
    return mem.cypher("""
MATCH (e:Event {type: "order_routed"})
-[:INVOLVES]->(w:Warehouse)
WHERE e.ts > datetime() - duration({days: 1})
RETURN w.name AS warehouse,
count(e) AS orders_today
ORDER BY orders_today DESC
""")

def supplier_restock_frequency(supplier_name):
    return mem.cypher("""
MATCH (e:Event {type: "restock_triggered"})
-[:INVOLVES]->(s:Supplier {name: $s})
RETURN count(e) AS total_restocks,
min(e.ts) AS first,
max(e.ts) AS last
""", s=supplier_name)

if __name__ == "__main__":
    store_order_routed("ORD_001", "SKU_001", "WH_BKK")
    store_restock("SKU_001", "Supplier_TH", 500)
    print(warehouse_load_today())
    mem.close()

```

Key Insight: The graph links Orders→Products→Warehouses→Suppliers. When a product keeps triggering restocks, agents can traverse the graph to identify the root cause: supplier lead time vs. demand spike — enabling predictive inventory management.

Part C — Challenges

C2.1

Add a `SupplierReliabilityAgent` that queries the KG for suppliers with >3 restocks this month and scores them by average lead time.

C2.2	Extend the LangGraph workflow to add a fraud check node that calls <code>recall_related("Order", order_id)</code> and flags duplicate orders.
C2.3	Build a GraphRAG chain: "Which product SKUs have caused the most warehouse rerouting events this quarter?"

Scenario Brief

A busy ED receives 200+ patients per day. A MAS must triage incoming patients by severity, assign them to available specialists, monitor bed capacity in real-time, and escalate critical cases. The KG tracks patient history, treatment outcomes, and resource utilisation to improve future triage accuracy — while maintaining HIPAA-compliant audit trails.

Agent Roles

Agent Role	Goal	Key Tool / Action
TriageAgent	Assess patient severity (ESI 1-5)	Symptom classification model
ResourceAgent	Monitor bed availability and staff roster	Hospital HIS/ADT system
SpecialistAgent	Assign patient to appropriate specialist	Skills registry + availability
AuditAgent	Record all decisions to KG for compliance	AgentMemory (HIPAA log)

Knowledge Graph Schema

■ Cypher schema (run once to set up)

```
// Hospital ED KG Schema
CREATE CONSTRAINT IF NOT EXISTS FOR (p:Patient)
  REQUIRE p.id IS UNIQUE;
CREATE CONSTRAINT IF NOT EXISTS FOR (b:Bed)
  REQUIRE b.id IS UNIQUE;

// Index for fast severity lookups
CREATE INDEX IF NOT EXISTS FOR (e:Event)
  ON (e.type);

// Sample domain data
MERGE (:Bed {id:"ED_01", zone:"critical", status:"free"})
MERGE (:Bed {id:"ED_02", zone:"general", status:"free"})
MERGE (:Specialist {id:"DR_001", specialty:"cardiology",
  available: true})

// Triage relationship template
// (:Patient)-[:ASSIGNED_TO]->(:Bed)
// (:Patient)-[:SEEN_BY]->(:Specialist)
// (:Event)-[:ABOUT]->(:Patient)
```

Part A — MAS Core Implementation

■ lab3_hospital_mas.py — Google ADK sequential pipeline

```
# lab3_hospital_mas.py — Google ADK
from google.adk.agents import Agent, SequentialAgent
from google.adk.tools import FunctionTool

# --- Tool functions ---
def assess_patient(patient_id: str,
  symptoms: list[str],
  vitals: dict) -> dict:
  """ESI triage scoring (1=critical, 5=minor)."""
  # Simplified scoring logic
```

```

critical = ["chest pain", "stroke", "sepsis", "trauma"]
esi = 1 if any(s in symptoms for s in critical) else 3
return {"patient_id": patient_id,
        "esi_score": esi,
        "priority": "CRITICAL" if esi <= 2 else "ROUTINE"}

def find_available_bed(zone: str) -> dict:
    """Find first available bed in zone."""
    # Replace with HIS API call
    beds = {"critical": "ED_01", "general": "ED_02"}
    return {"bed_id": beds.get(zone, "ED_WAIT"),
            "zone": zone}

def assign_specialist(specialty: str) -> dict:
    """Find available specialist."""
    return {"doctor": "DR_001",
            "specialty": specialty,
            "eta_minutes": 5}

# --- Agents ---
triage_agent = Agent(
    name="triage_agent",
    model="gemini-2.0-flash",
    instruction=(
        "You are an ED triage nurse. Use assess_patient "
        "to score severity. For ESI 1-2, assign critical bed. "
        "For ESI 3-5, assign general bed."
    ),
    tools=[
        FunctionTool(assess_patient),
        FunctionTool(find_available_bed),
    ]
)

specialist_agent = Agent(
    name="specialist_agent",
    model="gemini-2.0-flash",
    instruction=(
        "Based on patient condition, assign an appropriate "
        "specialist using assign_specialist tool."
    ),
    tools=[FunctionTool(assign_specialist)]
)

# Sequential pipeline: triage -> specialist
ed_pipeline = SequentialAgent(
    name="ed_pipeline",
    sub_agents=[triage_agent, specialist_agent]
)

if __name__ == "__main__":
    # Simulate with ADK runner
    from google.adk.runners import Runner
    from google.adk.sessions import InMemorySessionService
    session_svc = InMemorySessionService()
    runner = Runner(
        agent=ed_pipeline,
        app_name="HospitalED",
        session_service=session_svc
    )
    print("ED pipeline ready. Use runner.run() to process.")

```

■ lab3_hospital_memory.py — HIPAA audit trail in KG

```

# lab3_hospital_memory.py
from agent_memory import AgentMemory

mem = AgentMemory("HospitalED_AuditAgent")

def store_triage(patient_id, esi, symptoms, bed_id):
    """HIPAA-compliant triage record in KG."""
    mem.store_event(
        event_type="triage_completed",
        data={
            "esi": esi,
            "symptoms": symptoms,
            "bed_assigned": bed_id
        },
        entities=[
            ("Patient", patient_id),
            ("Bed", bed_id)
        ]
    )

def store_outcome(patient_id, diagnosis, outcome):
    mem.store_event(
        event_type="patient_outcome",
        data={"diagnosis": diagnosis, "outcome": outcome},
        entities=[("Patient", patient_id)]
    )

# --- Clinical analytics ---
def critical_patients_today():
    return mem.cypher("""
MATCH (e:Event {type:"triage_completed"})
WHERE e.ts > datetime() - duration({hours:12})
AND apoc.convert.fromJsonMap(e.data).esi <= 2
RETURN count(e) AS critical_count
""")

def patient_history(patient_id):
    """Full KG traversal for one patient."""
    return mem.cypher("""
MATCH (e:Event)-[:INVOLVES]->(p:Patient {name:$pid})
RETURN e.type AS event, e.ts AS time,
e.data AS details
ORDER BY e.ts
""", pid=patient_id)

def bed_utilisation():
    return mem.cypher("""
MATCH (e:Event {type:"triage_completed"})
-[:INVOLVES]->(b:Bed)
WHERE e.ts > datetime() - duration({hours:24})
RETURN b.name AS bed,
count(e) AS assignments
ORDER BY assignments DESC
""")

if __name__ == "__main__":
    store_triage("P_20240519_001", 2,
["chest pain","diaphoresis"], "ED_01")
    print("Critical today:", critical_patients_today())

```

```
mem.close()
```

Key Insight: Every triage decision is stored as an immutable Event node linked to Patient and Bed. Auditors can reconstruct the exact sequence of care decisions with Cypher path queries — satisfying HIPAA audit requirements without a separate log system.

Part C — Challenges

C3.1	Add a PredictiveTriageAgent that queries the KG for past patients with similar symptom profiles and returns the most likely diagnosis.
C3.2	Extend the KG schema with (:Medication)-[:PRESCRIBED_TO]->(:Patient) and detect drug interaction risks via graph traversal.
C3.3	Build a dashboard query: "Average time from triage to specialist assignment for ESI-2 patients this week."

Scenario Brief

A PCB factory produces 5,000 boards/day with 3 inspection stations (AOI, X-ray, final visual). Defective boards must be isolated, traced back to specific equipment and batches, and root-cause analysis performed automatically. The KG links defects to components, suppliers, and machine parameters to find systemic failure patterns.

Agent Roles

Agent Role	Goal	Key Tool / Action
InspectionAgent	Run AOI/X-ray and classify defects	CV model (YOLO defect detector)
TraceabilityAgent	Link defect to batch, machine, operator	MES / ERP lookup
RootCauseAgent	Analyse defect patterns across KG	GraphRAG + statistical analysis
QualityReportAgent	Generate SPC charts and CAPA reports	Report generator

Knowledge Graph Schema

■ Cypher schema (run once to set up)

```
// PCB QC Knowledge Graph
CREATE CONSTRAINT IF NOT EXISTS FOR (b:Batch)
  REQUIRE b.id IS UNIQUE;
CREATE CONSTRAINT IF NOT EXISTS FOR (p:PCBUnit)
  REQUIRE p.serial IS UNIQUE;

MERGE (:Machine {id:"AOI_01", type:"AOI",
  model:"OMRON VT-S730"})
MERGE (:Supplier {id:"SUP_001", name:"KyoceraTH",
  component:"Capacitor_100uF"})
MERGE (:DefectType {code:"OPEN_CIRCUIT",
  severity:"critical"})

// Relationships
// (:PCBUnit)-[:PRODUCED_IN]->(:Batch)
// (:PCBUnit)-[:INSPECTED_BY]->(:Machine)
// (:PCBUnit)-[:HAS_DEFECT]->(:DefectType)
// (:DefectType)-[:LINKED_TO]->(:Supplier)
```

Part A — MAS Core Implementation

■ lab4_pcb_mas.py — CrewAI QC pipeline

```
# lab4_pcb_mas.py - CrewAI with QC workflow
from crewai import Agent, Task, Crew, Process
from crewai_tools import BaseTool

class AOITool(BaseTool):
    name: str = "aoi_inspect"
    description: str = "Run AOI inspection on PCB serial"
    def _run(self, serial: str) -> dict:
        import random
        defects = ["OPEN_CIRCUIT", "SOLDER_BRIDGE",
        "MISSING_COMP", None]
        d = random.choice(defects)
```

```

return {"serial": serial,
        "defect": d,
        "pass": d is None,
        "confidence": 0.97}

class TraceTool(BaseTool):
    name: str = "trace_batch"
    description: str = "Look up batch and machine for serial"
    def _run(self, serial: str) -> dict:
        return {
            "batch_id": "BATCH_20240519_A",
            "machine_id": "AOI_01",
            "operator": "OP_007",
            "solder_paste_lot": "LOT_2024_88"
        }

class CAPATool(BaseTool):
    name: str = "create_capa"
    description: str = "Create Corrective Action report"
    def _run(self, defect_type: str,
              root_cause: str) -> str:
        return (f"CAPA-{defect_type[:4].upper()}: "
                f"Root cause: {root_cause}. "
                f"Action: Review {root_cause} parameters.")

inspector = Agent(
    role="AOI Inspector",
    goal="Detect and classify PCB defects with >99% accuracy",
    backstory="Computer vision QC specialist",
    tools=[AOITool()], verbose=False
)
tracer = Agent(
    role="Traceability Analyst",
    goal="Link every defect to its source batch and machine",
    backstory="Manufacturing MES expert",
    tools=[TraceTool()], verbose=False
)
rcaAgent = Agent(
    role="Root Cause Analyst",
    goal="Determine systemic root cause of defect patterns",
    backstory="Six Sigma Black Belt with graph analytics skills",
    tools=[CAPATool()], verbose=False
)

inspect_task = Task(
    description="Inspect PCBs: SN_001 through SN_010. "
    "List all defects found.",
    expected_output="List of defective serials with defect type",
    agent=inspector
)
trace_task = Task(
    description="For each defective PCB, retrieve batch "
    "and machine information.",
    expected_output="Traceability map: serial->batch->machine",
    agent=tracer
)
capa_task = Task(
    description="Analyse defect patterns. If >20% open circuits, "
    "the root cause is likely solder paste lot. "
    "Create CAPA report.",
    expected_output="CAPA report with root cause and actions",
    agent=rcaAgent
)

```

```

crew = Crew(
    agents=[inspector, tracer, rcaAgent],
    tasks=[inspect_task, trace_task, capa_task],
    process=Process.sequential, verbose=False
)

if __name__ == "__main__":
    result = crew.kickoff()
    print(result)

```

Part B — Neo4j Knowledge Graph Memory

■ lab4_pcb_memory.py — Defect traceability KG

```

# lab4_pcb_memory.py
from agent_memory import AgentMemory

mem = AgentMemory("PCB_QualityMAS")

def store_inspection(serial, defect, batch, machine):
    mem.store_event(
        event_type="pcb_inspection",
        data={"serial": serial, "defect": defect,
              "pass": defect is None},
        entities=[
            ("PCBUnit", serial),
            ("Batch", batch),
            ("Machine", machine)
        ]
    )
    if defect:
        mem.cypher("""
MATCH (p:PCBUnit {name: $serial})
MERGE (d:DefectType {code: $defect})
MERGE (p)-[:HAS_DEFECT {ts: datetime()}]->(d)
""", serial=serial, defect=defect)

# --- Root cause queries ---
def defect_by_machine():
    return mem.cypher("""
MATCH (e:Event {type:"pcb_inspection"})
-[:INVOLVES]->(m:Machine)
WHERE apoc.convert.fromJsonMap(e.data).pass = false
RETURN m.name AS machine,
count(e) AS defect_count
ORDER BY defect_count DESC
""")

def defect_by_batch():
    return mem.cypher("""
MATCH (e:Event {type:"pcb_inspection"})
-[:INVOLVES]->(b:Batch)
WHERE apoc.convert.fromJsonMap(e.data).pass = false
RETURN b.name AS batch,
count(e) AS defects,
collect(DISTINCT
apoc.convert.fromJsonMap(e.data).defect
) AS defect_types
ORDER BY defects DESC
""")

def supplier_defect_correlation():

```

```

"""Multi-hop: Defect → PCB → Batch → Supplier."""
return mem.cypher("""
MATCH (d:DefectType)-[:HAS_DEFECT]-(p:PCBUnit)
<-[:CONTAINS]-(b:Batch)
<-[:SUPPLIED]-(s:Supplier)
RETURN s.name AS supplier,
d.code AS defect,
count(p) AS affected_units
ORDER BY affected_units DESC LIMIT 10
""")

if __name__ == "__main__":
    store_inspection("SN_001", "OPEN_CIRCUIT",
"BATCH_A", "AOI_01")
    store_inspection("SN_002", None,
"BATCH_A", "AOI_01")
    print("By machine:", defect_by_machine())
    mem.close()

```

Key Insight:

The multi-hop query `DefectType←PCBUnit←Batch←Supplier` reveals supplier-defect correlations that would require 3 JOINS in SQL but one MATCH path in Cypher. This is graph-native root cause analysis — the manufacturing equivalent of explainable AI.

Part C — Challenges	
C4.1	Add a <code>(:ComponentLot)-[:USED_IN]->(:Batch)</code> relationship and query which component lots appear in batches with >5% defect rate.
C4.2	Implement a predictive quality alert: if the last 3 batches from one machine all had <code>defect_rate>2%</code> , auto-create a maintenance ticket.
C4.3	Use LLM Graph Builder to ingest your inspection report PDFs and auto-build the defect knowledge graph.

Scenario Brief

A digital bank processes 500,000 transactions/day. Fraud rings operate through chains of accounts. A MAS must detect anomalous transactions in <100ms, identify connected fraud rings via graph traversal, freeze accounts automatically, and generate SAR (Suspicious Activity Report) evidence. The KG is the core data structure — fraud rings only become visible as graph patterns.

Agent Roles

Agent Role	Goal	Key Tool / Action
TransactionMonitorAgent	Flag transactions $>3\sigma$ from baseline	Real-time transaction stream
FraudGraphAgent	Traverse KG for ring patterns	Neo4j GDS (graph algorithms)
AlertAgent	Freeze account + notify compliance	Core banking API + SMS
SARAgent	Generate Suspicious Activity Report	Regulatory template engine

Knowledge Graph Schema

■ Cypher schema (run once to set up)

```
// Fraud Detection KG
CREATE CONSTRAINT IF NOT EXISTS FOR (a:Account)
  REQUIRE a.id IS UNIQUE;
CREATE CONSTRAINT IF NOT EXISTS FOR (t:Transaction)
  REQUIRE t.id IS UNIQUE;

CREATE INDEX IF NOT EXISTS FOR (t:Transaction)
  ON (t.amount, t.ts);

// Graph algorithm support
CALL gds.graph.project.cypher(
  "fraud_graph",
  "MATCH (a:Account) RETURN id(a) AS id",
  "MATCH (a1:Account)-[:SENT]->(t:Transaction)
  -[:RECEIVED_BY]->(a2:Account)
  RETURN id(a1) AS source, id(a2) AS target,
  t.amount AS weight"
) YIELD graphName;

// Fraud ring detection via community detection
CALL gds.louvain.stream("fraud_graph")
YIELD nodeId, communityId
WITH communityId, collect(nodeId) AS members
WHERE size(members) > 3
RETURN communityId, members;
```

Part A — MAS Core Implementation

■ lab5_fraud_mas.py — LangGraph async fraud pipeline

```
# lab5_fraud_mas.py — LangGraph async workflow
import asyncio
from typing import TypedDict, List
from langgraph.graph import StateGraph, END

class FraudState(TypedDict):
```

```

tx_id: str
account_id: str
amount: float
risk_score: float
fraud_ring_detected: bool
actions_taken: List[str]

# --- Detection nodes ---
def score_transaction(state: FraudState):
    """Anomaly scoring: amount, velocity, location."""
    # Simplified: flag if >5000 or round number
    is_suspicious = (
        state["amount"] > 5000 or
        state["amount"] % 1000 == 0
    )
    risk = 0.85 if is_suspicious else 0.12
    return {**state, "risk_score": risk}

def check_fraud_ring(state: FraudState):
    """Query KG for connected suspicious accounts."""
    if state["risk_score"] < 0.5:
        return {**state, "fraud_ring_detected": False}

# In real implementation: call Neo4j GDS Louvain
# Here we simulate graph lookup
ring_found = state["risk_score"] > 0.8
return {**state, "fraud_ring_detected": ring_found,
        "actions_taken": state["actions_taken"]
        + ["graph_checked"]}

def freeze_account(state: FraudState):
    """Freeze account via core banking API."""
    print(f" [FREEZE] Account {state['account_id']}")
    return {**state,
            "actions_taken": state["actions_taken"]
            + [f"froze:{state['account_id']}"]}

def generate_sar(state: FraudState):
    """Generate SAR document."""
    print(f" [SAR] Transaction {state['tx_id']}, "
          f"amount={state['amount']}, "
          f"ring={state['fraud_ring_detected']}")
    return {**state,
            "actions_taken": state["actions_taken"]
            + ["sar_generated"]}

def pass_through(state: FraudState):
    return {**state, "actions_taken":
            state["actions_taken"] + ["approved"]}

# --- Build graph ---
wf = StateGraph(FraudState)
wf.add_node("score", score_transaction)
wf.add_node("ring_check", check_fraud_ring)
wf.add_node("freeze", freeze_account)
wf.add_node("sar", generate_sar)
wf.add_node("approve", pass_through)

wf.set_entry_point("score")
wf.add_edge("score", "ring_check")
wf.add_conditional_edges("ring_check",
    lambda s: "freeze" if s["fraud_ring_detected"]
    else ("sar" if s["risk_score"]>0.5

```

```

else "approve"),
{"freeze": "freeze", "sar": "sar", "approve": "approve"})
wf.add_edge("freeze", "sar")
wf.add_edge("sar", END)
wf.add_edge("approve", END)
app = wf.compile()

if __name__ == "__main__":
r = app.invoke({"tx_id": "TX_001",
"account_id": "ACC_777", "amount": 7000.00,
"risk_score": 0.0, "fraud_ring_detected": False,
"actions_taken": []})
print(r)

```

Part B — Neo4j Knowledge Graph Memory

■ lab5_fraud_memory.py — Fraud ring graph traversal

```

# lab5_fraud_memory.py
from agent_memory import AgentMemory

mem = AgentMemory("FraudDetectionMAS")

def store_transaction(tx_id, acc_id, amount, risk):
    mem.store_event(
        event_type="transaction_flagged",
        data={"tx_id": tx_id, "amount": amount,
              "risk_score": risk},
        entities=[
            ("Account", acc_id),
            ("Transaction", tx_id)
        ]
    )

def link_accounts(acc1, acc2, relationship):
    """Create direct account-to-account link in KG."""
    mem.cypher("""
    MERGE (a1:Account {name: $a1})
    MERGE (a2:Account {name: $a2})
    MERGE (a1)-[r:CONNECTED {type: $rel,
    ts: datetime()}]->(a2)
    """, a1=acc1, a2=acc2, rel=relationship)

# --- Fraud ring traversal ---
def find_ring_members(account_id, max_hops=3):
    """Find all accounts reachable within N hops."""
    return mem.cypher("""
    MATCH path = (start:Account {name: $acc})
    -[:CONNECTED*1..$hops]-(member:Account)
    RETURN DISTINCT member.name AS member,
    length(path) AS hops
    ORDER BY hops
    """, acc=account_id, hops=max_hops)

def high_risk_clusters():
    """Accounts with >3 flagged transactions in 1 hour."""
    return mem.cypher("""
    MATCH (e:Event {type:"transaction_flagged"})
    -[:INVOLVES]->(a:Account)
    WHERE e.ts > datetime() - duration({hours:1})
    AND apoc.convert.fromJsonMap(e.data).risk_score > 0.7
    RETURN a.name AS account,
    count(e) AS flags,

```

```

sum(apoc.convert.fromJsonMap(
e.data).amount) AS total_amount
ORDER BY flags DESC LIMIT 10
""")

if __name__ == "__main__":
store_transaction("TX_001", "ACC_777", 7000, 0.85)
link_accounts("ACC_777", "ACC_888", "PEER_TRANSFER")
print("Ring members:",
find_ring_members("ACC_777"))
mem.close()

```

Key Insight: Fraud rings are graph patterns, not row patterns. The `CONNECTED*1..3` Cypher path query finds all accounts within 3 hops — this is the exact technique banks use to uncover money laundering chains that span hundreds of transactions.

Part C — Challenges

C5.1	Implement the Louvain community detection call using Neo4j GDS: <code>CALL gds.louvain.stream("fraud_graph")</code> to automatically identify fraud clusters.
C5.2	Add a temporal analysis: flag accounts where the same amount is transferred in <5 minutes across 3 different accounts.
C5.3	Build a GraphRAG query: "Show me all accounts that sent money to ACC_777 in the last 7 days and their risk scores."

Scenario Brief

A city's 500 intersections are controlled by a MAS. During peak hours, emergency vehicles need priority corridors, public transit must stay on schedule, and CO2 sensors trigger green wave programmes. The KG stores the road network topology, historical congestion patterns, and incident causation chains to enable predictive signal optimisation.

Agent Roles

Agent Role	Goal	Key Tool / Action
SignalControlAgent	Adjust signal timing based on flow sensors	Traffic controller API
EmergencyAgent	Create priority corridor for emergency vehicles	Vehicle GPS + signal override
PredictionAgent	Forecast congestion using KG history	Historical KG query + ML model
TransitAgent	Prioritise bus lanes and tram crossings	GTFS real-time feed

Knowledge Graph Schema

■ Cypher schema (run once to set up)

```
// Smart City Traffic KG
CREATE CONSTRAINT IF NOT EXISTS FOR (i:Intersection)
  REQUIRE i.id IS UNIQUE;
CREATE CONSTRAINT IF NOT EXISTS FOR (r:RoadSegment)
  REQUIRE r.id IS UNIQUE;

// Road network topology
MERGE (i1:Intersection {id:"INT_001",
  lat:13.756, lon:100.501})
MERGE (i2:Intersection {id:"INT_002",
  lat:13.758, lon:100.503})
MERGE (i1)-[:ROAD {id:"SEG_001",
  length_m:250, lanes:4}]->(i2)

// Signal state history
// (:SignalEvent)-[:AT]->(:Intersection)
// (:Incident)-[:BLOCKED]->(:RoadSegment)
// (:EmergencyRoute)-[:THROUGH]->(:Intersection)
```

Part A — MAS Core Implementation

■ lab6_traffic_mas.py — Google ADK parallel agents

```
# lab6_traffic_mas.py — Google ADK multi-agent
from google.adk.agents import Agent, ParallelAgent
from google.adk.tools import FunctionTool

def get_flow_data(intersection_id: str) -> dict:
    """Read vehicle count from loop detectors."""
    import random
    return {
        "intersection": intersection_id,
        "vehicles_per_min": random.randint(10, 120),
        "occupancy_pct": random.randint(20, 95),
```

```

"avg_speed_kmh": random.randint(15, 80)
}

def set_signal_phase(intersection_id: str,
green_duration_s: int,
priority_direction: str) -> str:
print(f" [SIGNAL] {intersection_id}: "
f"green={green_duration_s}s, "
f"dir={priority_direction}")
return f"OK:{intersection_id}"

def create_emergency_corridor(
route: list[str]) -> dict:
"""Pre-clear intersections for emergency vehicle."""
cleared = []
for intersection in route:
# Force green on emergency path
cleared.append(intersection)
return {"corridor": cleared,
"estimated_clear_s": len(cleared) * 8}

signal_agent = Agent(
name="signal_agent",
model="gemini-2.0-flash",
instruction=(
"Monitor intersection flow. If occupancy > 80%, "
"extend green phase by 15s. Use set_signal_phase."
),
tools=[
FunctionTool(get_flow_data),
FunctionTool(set_signal_phase)
]
)

emergency_agent = Agent(
name="emergency_agent",
model="gemini-2.0-flash",
instruction=(
"When emergency vehicle detected, call "
"create_emergency_corridor with route."
),
tools=[FunctionTool(create_emergency_corridor)]
)

# Parallel: both agents run simultaneously
traffic_system = ParallelAgent(
name="traffic_system",
sub_agents=[signal_agent, emergency_agent]
)

if __name__ == "__main__":
from google.adk.runners import Runner
from google.adk.sessions import InMemorySessionService
runner = Runner(
agent=traffic_system,
app_name="SmartCityTraffic",
session_service=InMemorySessionService()
)
print("Traffic system ready.")

```

■ lab6_traffic_memory.py — City traffic KG

```
# lab6_traffic_memory.py
from agent_memory import AgentMemory

mem = AgentMemory("CityTrafficMAS")

def store_signal_event(intersection, phase, flow):
    mem.store_event(
        event_type="signal_change",
        data={"phase": phase, "flow": flow},
        entities=[("Intersection", intersection)]
    )

def store_incident(segment_id, incident_type,
    severity):
    mem.store_event(
        event_type="traffic_incident",
        data={"segment": segment_id,
            "type": incident_type,
            "severity": severity},
        entities=[("RoadSegment", segment_id)]
    )

# --- Predictive analytics ---
def congestion_forecast(intersection_id, hour_of_day):
    """What was avg occupancy at this hour historically?"""
    return mem.cypher("""
MATCH (e:Event {type:"signal_change"})
-[:INVOLVES]->(i:Intersection
{name: $inter})
WHERE e.ts.hour = $hour
RETURN avg(apoc.convert.fromJsonMap(
e.data).flow) AS avg_flow,
count(e) AS data_points
""", inter=intersection_id, hour=hour_of_day)

def incident_hotspots():
    """Road segments with most incidents."""
    return mem.cypher("""
MATCH (e:Event {type:"traffic_incident"})
-[:INVOLVES]->(s:RoadSegment)
WHERE e.ts > datetime() - duration({days:30})
RETURN s.name AS segment,
count(e) AS incidents,
collect(DISTINCT
apoc.convert.fromJsonMap(e.data).type
) AS types
ORDER BY incidents DESC LIMIT 10
""")

def emergency_route_history():
    return mem.cypher("""
MATCH (e:Event {type:"emergency_corridor"})
RETURN e.ts AS time,
apoc.convert.fromJsonMap(e.data).route
AS route
ORDER BY e.ts DESC LIMIT 5
""")

if __name__ == "__main__":
    store_signal_event("INT_001", "NS_GREEN", 87)
    store_incident("SEG_001", "ACCIDENT", "HIGH")
    print("Hotspots:", incident_hotspots())
```

```
mem.close()
```

Key Insight: Historical congestion patterns stored in the KG let agents predict tomorrow's peak load using yesterday's graph data. The road network topology is itself a graph — Neo4j is the natural fit for routing algorithms on the actual road structure.

Part C — Challenges

C6.1	Implement shortest-path emergency routing using <code>CALL gds.shortestPath.dijkstra.stream()</code> on the road network graph.
C6.2	Add a <code>CO2SensorAgent</code> that triggers green wave when a road segment records >400ppm CO2 for 5 consecutive readings.
C6.3	Use the KG to identify which intersections appear most in emergency corridors and proactively optimise their baseline timing.

Scenario Brief

A SaaS company handles 2,000 support tickets/day. A MAS classifies tickets, searches the knowledge base, generates responses, and escalates to humans only when confidence is low. The KG stores solution histories — so agents learn that "error 503 on /api/v2/export" was always caused by storage quota and auto-resolves it next time.

Agent Roles

Agent Role	Goal	Key Tool / Action
TriageAgent	Classify ticket type and urgency	NLP classifier (GLM-4)
KnowledgeAgent	Search KB + KG for similar past tickets	Vector search + GraphRAG
ResponseAgent	Draft personalised reply	LLM (z.ai GLM-4.5-air)
EscalationAgent	Route to human if confidence <0.7	Zendesk / Freshdesk API

Knowledge Graph Schema

■ Cypher schema (run once to set up)

```
// Customer Support KG
CREATE CONSTRAINT IF NOT EXISTS FOR (c:Customer)
  REQUIRE c.id IS UNIQUE;
CREATE CONSTRAINT IF NOT EXISTS FOR (t:Ticket)
  REQUIRE t.id IS UNIQUE;
CREATE CONSTRAINT IF NOT EXISTS FOR (s:Solution)
  REQUIRE s.id IS UNIQUE;

MERGE (:Product {name:"CERO_API",
  version:"v2.3"})
MERGE (:ErrorCode {code:"ERR_503",
  description:"Storage quota exceeded"})

// Resolution graph
// (:Ticket)-[:RESOLVED_BY]->(:Solution)
// (:Solution)-[:APPLIES_TO]->(:ErrorCode)
// (:Customer)-[:SUBMITTED]->(:Ticket)
// (:Solution)-[:RELATED_TO]->(:Solution)
```

Part A — MAS Core Implementation

■ lab7_support_mas.py — CrewAI support pipeline

```
# lab7_support_mas.py - CrewAI CX pipeline
from crewai import Agent, Task, Crew, Process
from crewai_tools import BaseTool

class KBSearchTool(BaseTool):
    name: str = "kb_search"
    description: str = "Search knowledge base articles"
    def _run(self, query: str) -> list:
        # Replace with real vector search
        articles = {
            "503": "Solution: Clear storage, upgrade plan",
            "login": "Solution: Reset OAuth token",
```

```

"slow": "Solution: Enable CDN caching"
}
for k, v in articles.items():
    if k.lower() in query.lower():
        return [{"article": v, "score": 0.92}]
    return [{"article": "No match found", "score": 0.1}]

class DraftResponseTool(BaseTool):
    name: str = "draft_response"
    description: str = "Draft customer reply using solution"
    def _run(self, solution: str,
            customer_name: str) -> str:
        return (f"Hi {customer_name}, thank you for "
                f"reaching out. {solution} "
                f"Let us know if this helps!")

class EscalateTool(BaseTool):
    name: str = "escalate_ticket"
    description: str = "Escalate to human agent"
    def _run(self, ticket_id: str,
            reason: str) -> str:
        print(f" [ESCALATE] {ticket_id}: {reason}")
        return f"Escalated to Tier-2 team"

triage = Agent(
    role="Ticket Triage Specialist",
    goal="Classify ticket urgency and category accurately",
    backstory="Support operations analyst",
    verbose=False
)
knowledge = Agent(
    role="Knowledge Agent",
    goal="Find best solution from KB and past tickets",
    backstory="Support engineer with 5yr product experience",
    tools=[KBSearchTool()], verbose=False
)
responder = Agent(
    role="Response Writer",
    goal="Write empathetic, accurate customer replies",
    backstory="Customer success communication expert",
    tools=[DraftResponseTool(), EscalateTool()],
    verbose=False
)

classify_task = Task(
    description="Ticket: 'Getting 503 error on API export'. "
    "Classify: type=API_ERROR, urgency=HIGH, "
    "product=CERO_API",
    expected_output="Classification dict",
    agent=triage
)
search_task = Task(
    description="Search KB for 503 API error solution. "
    "Return top article.",
    expected_output="Solution article with confidence",
    agent=knowledge
)
respond_task = Task(
    description="Draft reply to customer 'Somchai'. "
    "If confidence < 0.7, escalate instead.",
    expected_output="Final response or escalation notice",
    agent=responder
)

```

```

crew = Crew(
    agents=[triage, knowledge, responder],
    tasks=[classify_task, search_task, respond_task],
    process=Process.sequential, verbose=False
)

if __name__ == "__main__":
    result = crew.kickoff()
    print(result)

```

Part B — Neo4j Knowledge Graph Memory

■ lab7_support_memory.py — Solution-error KG

```

# lab7_support_memory.py
from agent_memory import AgentMemory

mem = AgentMemory("SupportMAS")

def store_ticket_resolved(ticket_id, customer_id,
    error_code, solution_id,
    resolution_time_min):
    mem.store_event(
        event_type="ticket_resolved",
        data={"ticket": ticket_id,
            "error": error_code,
            "solution": solution_id,
            "time_min": resolution_time_min},
        entities=[
            ("Ticket", ticket_id),
            ("Customer", customer_id),
            ("ErrorCode", error_code),
            ("Solution", solution_id)
        ]
    )

# Link solution to error code for future lookup
mem.cypher("""
MERGE (s:Solution {name: $sol})
MERGE (e:ErrorCode {code: $err})
MERGE (s)-[:RESOLVES {count:1}]->(e)
ON MATCH SET (s)-[:RESOLVES]->(e).count =
(s)-[:RESOLVES]->(e).count + 1
""", sol=solution_id, err=error_code)

def find_solution_for_error(error_code):
    """What solution has resolved this error most?"""
    return mem.cypher("""
MATCH (s:Solution)-[r:RESOLVES]->
(e:ErrorCode {code: $err})
RETURN s.name AS solution,
r.count AS times_used
ORDER BY r.count DESC LIMIT 3
""", err=error_code)

def customer_ticket_history(customer_id):
    return mem.cypher("""
MATCH (e:Event {type:"ticket_resolved"})
-[:INVOLVES]->(c:Customer {name: $cid})
RETURN e.data AS details, e.ts AS time
ORDER BY e.ts DESC LIMIT 10
""", cid=customer_id)

```

```
def avg_resolution_time_by_error():
    return mem.cypher("""
MATCH (e:Event {type:"ticket_resolved"})
RETURN apoc.convert.fromJsonMap(e.data).error
AS error_code,
avg(apoc.convert.fromJsonMap(
e.data).time_min) AS avg_min,
count(e) AS ticket_count
ORDER BY avg_min DESC
""")

if __name__ == "__main__":
    store_ticket_resolved("TKT_001", "C_001",
"ERR_503", "SOL_STORAGE_UPGRADE", 8)
    print("Solution for ERR_503:",
    find_solution_for_error("ERR_503"))
    mem.close()
```

Key Insight: The KG builds a solution graph over time. Each resolved ticket strengthens the Solution→ErrorCode edge. New tickets are resolved by traversing this graph — no vector search needed for known errors. This is how support systems achieve 90%+ auto-resolution.

Part C — Challenges

C7.1	Add a "similar tickets" feature: when a new ticket arrives, find the 3 most similar past tickets using MATCH (t:Ticket)-[:RESOLVED_BY]->(s:Solution)-[:RESOLVES]->(e:ErrorCode).
C7.2	Implement a feedback loop: customers rate solutions 1-5. Store ratings in the KG and weight solution recommendations by rating.
C7.3	Build a GraphRAG query: "What are the top 3 most common issues for enterprise customers this month?"

Scenario Brief

A manufacturer sources 500+ components from 80 suppliers across 15 countries. When a geopolitical event, port closure, or supplier bankruptcy disrupts supply, a MAS must identify which production lines are at risk, find alternative suppliers, re-route shipments, and update customer delivery commitments — all within 2 hours. The KG maps the entire supplier-component-product dependency graph.

Agent Roles

Agent Role	Goal	Key Tool / Action
DisruptionMonitorAgent	Watch news/APIs for supply chain events	RSS feeds + geopolitical risk API
ImpactAnalysisAgent	Map disruption to affected components	KG traversal (supplier→component→product)
AlternativeSourceAgent	Find qualified alternative suppliers	Supplier qualification DB + KG
CommitmentAgent	Update delivery dates and notify customers	ERP + CRM integration

Knowledge Graph Schema

■ Cypher schema (run once to set up)

```
// Supply Chain KG
CREATE CONSTRAINT IF NOT EXISTS FOR (s:Supplier)
  REQUIRE s.id IS UNIQUE;
CREATE CONSTRAINT IF NOT EXISTS FOR (c:Component)
  REQUIRE c.id IS UNIQUE;
CREATE CONSTRAINT IF NOT EXISTS FOR (p:Product)
  REQUIRE p.id IS UNIQUE;

MERGE (:Supplier {id:"SUP_CN_01",
  name:"Guangzhou_Parts",
  country:"CN", risk_score:0.6})
MERGE (:Component {id:"COMP_MCU",
  name:"STM32_MCU",
  lead_time_days:45})
MERGE (:Product {id:"PROD_CERO_HW",
  name:"CERO_Controller"})

// Dependencies
MATCH (s:Supplier {id:"SUP_CN_01"}),
  (c:Component {id:"COMP_MCU"})
MERGE (s)-[:SUPPLIES {
  qty_per_month:2000, sole_source:true}]->(c);
MATCH (c:Component {id:"COMP_MCU"}),
  (p:Product {id:"PROD_CERO_HW"})
MERGE (c)-[:REQUIRED_FOR {qty_per_unit:1}]->(p);
```

Part A — MAS Core Implementation

■ lab8_supply_mas.py — CrewAI disruption response

```
# lab8_supply_mas.py – CrewAI supply chain response
from crewai import Agent, Task, Crew, Process
from crewai_tools import BaseTool

class DisruptionTool(BaseTool):
```

```

name: str = "check_disruptions"
description: str = "Check active supply disruptions"
def _run(self, region: str) -> list:
# Replace with real news/risk API
if region == "CN":
return [{"event": "Port closure",
"location": "Shanghai",
"severity": "HIGH",
"duration_days": 14}]
return []

class ImpactTool(BaseTool):
name: str = "assess_impact"
description: str = "Which components affected by event?"
def _run(self, supplier_id: str) -> dict:
# Replace with KG traversal
return {"affected_components": ["COMP_MCU"],
"affected_products": ["PROD_CERO_HW"],
"at_risk_units": 500,
"sole_source": True}

class AltSupplierTool(BaseTool):
name: str = "find_alternatives"
description: str = "Find alternative suppliers"
def _run(self, component_id: str) -> list:
return [
{"supplier": "TW_Chips_01",
"country": "TW",
"lead_time_days": 60,
"qualification": "APPROVED"},
{"supplier": "JP_Semi_02",
"country": "JP",
"lead_time_days": 45,
"qualification": "PENDING"}
]

monitor = Agent(
role="Disruption Monitor",
goal="Identify active supply chain disruptions",
backstory="Global logistics risk analyst",
tools=[DisruptionTool()], verbose=False
)
impact_analyst = Agent(
role="Impact Analyst",
goal="Map disruptions to production impact",
backstory="Supply chain engineer with full BOM knowledge",
tools=[ImpactTool()], verbose=False
)
sourcing = Agent(
role="Alternative Sourcing Agent",
goal="Find qualified alternative suppliers fast",
backstory="Procurement specialist with global supplier network",
tools=[AltSupplierTool()], verbose=False
)

t1 = Task(description="Check disruptions in CN region.",
expected_output="List of active disruptions",
agent=monitor)
t2 = Task(description="For Shanghai port closure, assess "
"impact on our supplier SUP_CN_01.",
expected_output="Impact assessment with at-risk units",
agent=impact_analyst)
t3 = Task(description="Find alternatives for COMP_MCU. "

```

```

"Rank by lead time and qualification.",
expected_output="Ranked alternative supplier list",
agent=sourcing)

crew = Crew(agents=[monitor, impact_analyst, sourcing],
tasks=[t1, t2, t3],
process=Process.sequential, verbose=False)

if __name__ == "__main__":
print(crew.kickoff())

```

Part B — Neo4j Knowledge Graph Memory

■ lab8_supply_memory.py — Supplier dependency KG

```

# lab8_supply_memory.py
from agent_memory import AgentMemory

mem = AgentMemory("SupplyChainMAS")

def store_disruption(supplier_id, event_type,
severity, duration_days):
mem.store_event(
event_type="supply_disruption",
data={"event": event_type,
"severity": severity,
"duration_days": duration_days},
entities=[("Supplier", supplier_id)]
)

def store_alternative_selected(component_id,
old_supplier,
new_supplier):
mem.store_event(
event_type="supplier_switch",
data={"component": component_id,
"from": old_supplier,
"to": new_supplier},
entities=[
("Component", component_id),
("Supplier", new_supplier)
]
)

# --- Risk analytics ---
def single_source_risk():
"""Find components with only 1 approved supplier."""
return mem.cypher("""
MATCH (c:Component)<-[:SUPPLIES]-(s:Supplier)
WITH c, count(s) AS supplier_count
WHERE supplier_count = 1
RETURN c.name AS component,
supplier_count
ORDER BY c.name
""")

def disruption_history_by_country(country):
return mem.cypher("""
MATCH (e:Event {type:"supply_disruption"})
-[:INVOLVES]->(s:Supplier {country: $c})
RETURN s.name AS supplier,
e.ts AS time,
apoc.convert.fromJsonMap(e.data).event

```

```

AS event_type
ORDER BY e.ts DESC
"", c=country)

def cascade_impact(supplier_id):
    """Full dependency traversal: supplier → products."""
    return mem.cypher("""
MATCH path = (s:Supplier {name: $sid})
-[:SUPPLIES]->(c:Component)
-[:REQUIRED_FOR]->(p:Product)
RETURN s.name AS supplier,
c.name AS component,
p.name AS product,
length(path) AS depth
ORDER BY depth
"", sid=supplier_id)

if __name__ == "__main__":
    store_disruption("SUP_CN_01",
"PORT_CLOSURE", "HIGH", 14)
    print("Single-source risk:", single_source_risk())
    mem.close()

```

Key Insight: The cascade_impact() Cypher traversal shows the full Supplier→Component→Product chain in one query — the equivalent of a Bill of Materials explosion. When combined with disruption history, agents can quantify revenue-at-risk per disruption event.

Part C — Challenges

C8.1	Add a RiskScoreAgent that updates each Supplier node's risk_score based on disruption frequency: suppliers with >2 disruptions/year get score >0.7.
C8.2	Implement dual-sourcing recommendations: for each sole-source component, automatically search for and add a second qualified supplier.
C8.3	Use LLM Graph Builder to ingest supplier qualification PDFs and auto-extract Supplier→Certification→Standard relationships.

Scenario Brief

A real estate fund evaluates 50 properties/month for investment. Due diligence requires analysing financials, comparable sales, zoning regulations, environmental risks, and macro trends. A MAS automates research, extracts data from PDFs and web sources into a KG, and produces an investment recommendation with risk scores in under 2 hours instead of 2 weeks.

Agent Roles

Agent Role	Goal	Key Tool / Action
PropertyAnalysisAgent	Extract NOI, cap rate, occupancy from docs	PDF extraction + LLM parsing
MarketResearchAgent	Fetch comparable sales and market trends	Property database API
RiskAssessmentAgent	Score flood, seismic, regulatory risks	Government data APIs + KG
InvestmentReportAgent	Synthesise findings into IC memo	Template engine + LLM

Knowledge Graph Schema

■ Cypher schema (run once to set up)

```
// Real Estate Due Diligence KG
CREATE CONSTRAINT IF NOT EXISTS FOR (p:Property)
  REQUIRE p.id IS UNIQUE;
CREATE CONSTRAINT IF NOT EXISTS FOR (l:Location)
  REQUIRE l.postcode IS UNIQUE;

MERGE (:Property {id:"PROP_001",
  address:"123 Sukhumvit Rd",
  type:"office", floors:12})
MERGE (:Location {postcode:"10110",
  district:"Wattana",
  city:"Bangkok"})
MERGE (:RiskFactor {type:"FLOOD",
  severity:"MEDIUM",
  return_period_yrs:50})

// Knowledge relationships
// (:Property)-[:LOCATED_IN]->(:Location)
// (:Property)-[:HAS_RISK]->(:RiskFactor)
// (:Property)-[:COMPARABLE_TO]->(:Property)
// (:Transaction)-[:FOR_PROPERTY]->(:Property)
```

Part A — MAS Core Implementation

■ lab9_realestate_mas.py — CrewAI due diligence

```
# lab9_realestate_mas.py - CrewAI DD pipeline
from crewai import Agent, Task, Crew, Process
from crewai_tools import BaseTool, PDFSearchTool

class ComparablesTool(BaseTool):
    name: str = "get_comparables"
    description: str = "Fetch comparable sales in area"
    def _run(self, location: str,
    property_type: str) -> list:
```

```

# Replace with real property DB API
return [
    {"address": "456 Sukhumvit",
     "sold_price_thb": 85_000_000,
     "cap_rate": 0.062,
     "date": "2024-03"},
    {"address": "789 Asok Rd",
     "sold_price_thb": 92_000_000,
     "cap_rate": 0.058,
     "date": "2024-01"},
]

class RiskScoreTool(BaseTool):
    name: str = "assess_risks"
    description: str = "Score property risks 0-10"
    def _run(self, property_id: str) -> dict:
        return {"flood_risk": 4.2,
                "seismic_risk": 1.5,
                "regulatory_risk": 2.8,
                "vacancy_risk": 3.1,
                "overall": 3.2}

analyst = Agent(
    role="Property Financial Analyst",
    goal="Extract and verify NOI, cap rate, occupancy",
    backstory="CFA with 10yr real estate experience",
    tools=[PDFSearchTool()], verbose=False
)
market_researcher = Agent(
    role="Market Research Agent",
    goal="Find comparable sales and validate valuation",
    backstory="Prop-tech data scientist",
    tools=[ComparablesTool()], verbose=False
)
risk_agent = Agent(
    role="Risk Assessment Agent",
    goal="Identify and score all investment risks",
    backstory="Real estate risk analyst",
    tools=[RiskScoreTool()], verbose=False
)
reporter = Agent(
    role="IC Memo Writer",
    goal="Write concise investment committee memo",
    backstory="Investment banking analyst",
    verbose=False
)

t1 = Task(description="Analyse PROP_001 financials. "
    "Extract: NOI, cap rate, occupancy.",
    expected_output="Financial metrics dict",
    agent=analyst)
t2 = Task(description="Find 5 comparable sales in Wattana "
    "district for office properties.",
    expected_output="Ranked comparables with cap rates",
    agent=market_researcher)
t3 = Task(description="Score PROP_001 on flood, seismic, "
    "regulatory, and vacancy risks.",
    expected_output="Risk score card",
    agent=risk_agent)
t4 = Task(description="Write 1-page IC memo for PROP_001: "
    "recommendation, valuation, risks.",
    expected_output="Investment Committee memo",
    agent=reporter)

```

```

crew = Crew(
    agents=[analyst, market_researcher, risk_agent,
            reporter],
    tasks=[t1, t2, t3, t4],
    process=Process.sequential, verbose=False
)

if __name__ == "__main__":
    print(crew.kickoff())

```

Part B — Neo4j Knowledge Graph Memory

■ lab9_realestate_memory.py — Property investment KG

```

# lab9_realestate_memory.py
from agent_memory import AgentMemory

mem = AgentMemory("RealEstateMAS")

def store_property_analysis(prop_id, financials,
                           risks):
    mem.store_event(
        event_type="dd_completed",
        data={**financials, **risks},
        entities=[("Property", prop_id)]
    )
    # Store risk relationships
    for risk_type, score in risks.items():
        if score > 3.0:
            mem.cypher("""
MERGE (p:Property {name: $pid})
MERGE (r:RiskFactor {type: $rtype,
score: $score})
MERGE (p)-[:HAS_RISK {
score: $score,
assessed: datetime()}]->(r)
""", pid=prop_id,
      rtype=risk_type.upper(),
      score=score)

def store_comparable(prop_id, comp_address,
                     cap_rate, price_thb):
    mem.cypher("""
MERGE (p:Property {name: $pid})
MERGE (c:Property {name: $addr})
SET c.cap_rate = $cap,
c.sold_price = $price
MERGE (p)-[:COMPARABLE_TO {
cap_rate_delta: abs($cap -
coalesce(p.cap_rate, 0.06))
}]->(c)
""", pid=prop_id, addr=comp_address,
      cap=cap_rate, price=price_thb)

# --- Investment analytics ---
def portfolio_risk_profile():
    return mem.cypher("""
MATCH (p:Property)-[r:HAS_RISK]->(rf:RiskFactor)
RETURN p.name AS property,
rf.type AS risk,
r.score AS score
ORDER BY r.score DESC LIMIT 20

```

```

"""
def best_cap_rate_comparables(min_cap=0.06):
    return mem.cypher("""
MATCH (p:Property)-[:COMPARABLE_TO]->(c:Property)
WHERE c.cap_rate >= $min
RETURN p.name AS subject,
       c.name AS comparable,
       c.cap_rate AS cap_rate
ORDER BY c.cap_rate DESC
""", min=min_cap)

if __name__ == "__main__":
    store_property_analysis(
        "PROP_001",
        {"noi_thb": 5_200_000, "cap_rate": 0.061},
        {"flood_risk": 4.2, "regulatory_risk": 2.8}
    )
mem.close()

```

Key Insight: Using LLM Graph Builder to ingest property PDFs (valuations, title deeds, environmental reports) automatically creates Property→RiskFactor→Regulation nodes. A portfolio manager can then ask: "Which properties are near flood zones and have cap rates below 5%?" — answered by one Cypher query.

Part C — Challenges

C9.1	Use LLM Graph Builder to process a property valuation PDF and auto-extract: Property→Location→Comparables relationships.
C9.2	Add a MacroTrendAgent that stores interest rate + GDP data as MacroEvent nodes linked to affected property types.
C9.3	Build a portfolio optimization query: find properties with cap_rate > 0.065 AND overall_risk < 3.5 AND city = "Bangkok".

Scenario Brief

A restaurant chain with 150 outlets must maintain FDA/HACCP compliance. IoT sensors monitor cold storage temperatures, ingredient expiry dates, and cleaning schedules. When violations are detected, a MAS must trigger corrective actions, record audit trails, trace contaminated ingredients back to suppliers, and generate regulatory reports automatically.

Agent Roles

Agent Role	Goal	Key Tool / Action
TemperatureMonitorAgent	Alert on cold chain violations (>5°C)	IoT sensor MQTT stream
IngredientTraceAgent	Trace ingredients from supplier to dish	Batch tracking system
ComplianceAgent	Check HACCP critical control points	HACCP ruleset engine
RegulatoryReportAgent	Generate FDA/health dept audit report	Regulatory template + KG query

Knowledge Graph Schema

■ Cypher schema (run once to set up)

```
// Food Safety KG
CREATE CONSTRAINT IF NOT EXISTS FOR (r:Restaurant)
  REQUIRE r.id IS UNIQUE;
CREATE CONSTRAINT IF NOT EXISTS FOR (i:Ingredient)
  REQUIRE i.lot_id IS UNIQUE;
CREATE CONSTRAINT IF NOT EXISTS FOR (s:Supplier)
  REQUIRE s.id IS UNIQUE;

MERGE (:Restaurant {id:"REST_001",
  name:"AltoKitchen_BKK",
  location:"Bangkok"})
MERGE (:Supplier {id:"SUP_MEAT_01",
  name:"FreshMeatTH",
  haccp_certified:true})
MERGE (:Ingredient {lot_id:"LOT_CHICKEN_20240519",
  type:"CHICKEN",
  expiry:"2024-05-23",
  storage_temp_max_c:4})

// Chain of custody
// (:Supplier)-[:DELIVERED]->(i:Ingredient)
// (:Restaurant)-[:RECEIVED]->(i:Ingredient)
// (:Ingredient)-[:USED_IN]->(d:Dish)
// (:ColdStorage)-[:STORES]->(i:Ingredient)
// (:Violation)-[:OCCURRED_AT]->(r:Restaurant)
```

Part A — MAS Core Implementation

■ lab10_foodsafety_mas.py — Google ADK parallel HACCP

```
# lab10_foodsafety_mas.py - Google ADK
from google.adk.agents import Agent, ParallelAgent
from google.adk.tools import FunctionTool

def check_storage_temps(restaurant_id: str) -> list:
    """Read all cold storage sensors."""
```

```

# Replace with real IoT API
return [
{"unit": "COLD_01", "temp_c": 3.2, "ok": True},
{"unit": "COLD_02", "temp_c": 6.8, "ok": False,
"violation": "ABOVE_5C_THRESHOLD"},
{"unit": "FREEZER_01", "temp_c": -18.1, "ok": True}
]

def check_ingredient_expiry(restaurant_id: str,
days_ahead: int = 1) -> list:
"""Find ingredients expiring within N days."""
from datetime import date, timedelta
cutoff = date.today() + timedelta(days=days_ahead)
# Replace with real inventory API
return [
{"lot": "LOT_SALMON_20240519",
"ingredient": "SALMON",
"expiry": str(cutoff),
"qty_kg": 12.5}
]

def create_violation_report(
restaurant_id: str,
violation_type: str,
details: dict) -> str:
report_id = f"VIO_{restaurant_id}_{violation_type}"
print(f" [VIOLATION] {report_id}: {details}")
return report_id

def send_corrective_action(
restaurant_id: str,
action: str) -> str:
print(f" [ACTION] {restaurant_id}: {action}")
return f"CA_SENT:{restaurant_id}"

temp_monitor = Agent(
name="temp_monitor",
model="gemini-2.0-flash",
instruction=(
"Check cold storage temps at all units. "
"Flag any unit above 5°C as a HACCP violation. "
"Create violation report and corrective action."
),
tools=[
FunctionTool(check_storage_temps),
FunctionTool(create_violation_report),
FunctionTool(send_corrective_action)
]
)

ingredient_agent = Agent(
name="ingredient_agent",
model="gemini-2.0-flash",
instruction=(
"Check for ingredients expiring within 24 hours. "
"Flag them and create disposal instructions."
),
tools=[
FunctionTool(check_ingredient_expiry),
FunctionTool(create_violation_report)
]
)

```

```
# Both monitors run in parallel
food_safety_system = ParallelAgent(
    name="food_safety_system",
    sub_agents=[temp_monitor, ingredient_agent]
)

if __name__ == "__main__":
    from google.adk.runners import Runner
    from google.adk.sessions import InMemorySessionService
    runner = Runner(
        agent=food_safety_system,
        app_name="FoodSafety",
        session_service=InMemorySessionService()
    )
    print("Food safety monitoring active.")
```

Part B — Neo4j Knowledge Graph Memory

■ lab10_foodsafety_memory.py — HACCP compliance KG

```
# lab10_foodsafety_memory.py
from agent_memory import AgentMemory

mem = AgentMemory("FoodSafetyMAS")

def store_temp_reading(restaurant_id, unit_id,
                      temp_c, is_violation):
    mem.store_event(
        event_type="temp_reading",
        data={"unit": unit_id, "temp_c": temp_c,
              "violation": is_violation},
        entities=[
            ("Restaurant", restaurant_id),
            ("ColdStorage", unit_id)
        ]
    )
    if is_violation:
        mem.cypher("""
MERGE (r:Restaurant {name: $rid})
MERGE (v:Violation {
    type: "COLD_CHAIN_BREACH",
    unit: $unit,
    temp_c: $temp
})
MERGE (v)-[:OCCURRED_AT {
    ts: datetime()}]->(r)
""", rid=restaurant_id,
      unit=unit_id, temp=temp_c)

def store_ingredient_trace.lot_id, supplier_id,
restaurant_id):
    mem.cypher("""
MERGE (i:Ingredient {name: $lot})
MERGE (s:Supplier {name: $sup})
MERGE (r:Restaurant {name: $rest})
MERGE (s)-[:DELIVERED]->(i)
MERGE (r)-[:RECEIVED {ts: datetime()}]->(i)
""", lot=lot_id, sup=supplier_id, rest=restaurant_id)

# --- Compliance analytics ---
def cold_chain_violations_30d():
    return mem.cypher("""
MATCH (e:Event {type:"temp_reading"})
```

```

WHERE e.ts > datetime() - duration({days:30})
AND apoc.convert.fromJsonMap(e.data).violation
= true
MATCH (e)-[:INVOLVES]->(r:Restaurant)
RETURN r.name AS restaurant,
count(e) AS violations,
max(apoc.convert.fromJsonMap(
e.data).temp_c) AS max_temp
ORDER BY violations DESC
""")

def trace_ingredient_exposure(lot_id):
    """Which restaurants received this ingredient lot?"""
    return mem.cypher("""
MATCH (i:Ingredient {name: $lot})
<-[:RECEIVED]-(r:Restaurant)
RETURN r.name AS restaurant,
i.name AS ingredient_lot
""", lot=lot_id)

def supplier_violation_correlation():
    """Link violations to ingredient supplier."""
    return mem.cypher("""
MATCH (s:Supplier)-[:DELIVERED]->(i:Ingredient)
<-[:RECEIVED]-(r:Restaurant)
<-[:OCCURRED_AT]-(v:Violation)
WHERE v.ts > datetime() - duration({days:90})
RETURN s.name AS supplier,
count(v) AS linked_violations
ORDER BY linked_violations DESC
""")

if __name__ == "__main__":
    store_temp_reading("REST_001", "COLD_02", 6.8, True)
    store_ingredient_trace("LOT_CHICKEN_001",
    "FreshMeatTH", "REST_001")
    print("Violations 30d:", cold_chain_violations_30d())
    mem.close()

```

Key Insight: The `trace_ingredient_exposure()` query answers the critical food safety question: "All restaurants that received Lot X." In a contamination incident, this graph traversal takes milliseconds vs. hours of manual spreadsheet tracing — enabling immediate recall notifications.

Part C — Challenges

C10.1	Add a RecallAgent: when an ingredient lot is flagged as contaminated, automatically notify all restaurants that received it using the KG traversal + SMS gateway.
C10.2	Build a HACCP audit report that queries: all violations by restaurant, all cold chain breaches by unit, and all ingredient lots received in the last 7 days.
C10.3	Implement predictive maintenance for cold storage: if a unit has had 3 temp violations in 7 days, auto-schedule inspection.

Capstone: Unified MAS + Knowledge Graph

Combine all memory types • GraphRAG pipeline • Deployment

After completing all 10 labs, you have 10 separate Neo4j schemas and AgentMemory instances. The capstone challenge: build a single UnifiedMemorySystem that spans all domains and enables cross-domain agent reasoning.

Unified Memory Architecture

Memory Layer	Implementation	Scope
L1: Working	Python dict / ADK session.state	Single agent run
L2: Episodic	SQLite events log	Agent lifetime (hours)
L3: Semantic	ChromaDB with text embeddings	Cross-session similarity
L4: Knowledge Graph	Neo4j (all 10 lab schemas merged)	Permanent structured memory

GraphRAG Pipeline

■ capstone_graphrag.py — Natural language → Cypher → Answer

```
# capstone_graphrag.py - Full GraphRAG pipeline
from langchain_community.graphs import Neo4jGraph
from langchain.chains import GraphCypherQAChain
from langchain_openai import ChatOpenAI

# z.ai compatible endpoint
llm = ChatOpenAI(
    base_url="https://api.z.ai/api/paas/v4/",
    model="glm-4.5-air",
    api_key="YOUR_Z_AI_KEY",
    temperature=0
)

graph = Neo4jGraph(
    url="bolt://localhost:7687",
    username="neo4j",
    password="mas_memory_2024"
)
graph.refresh_schema() # Auto-detect all node labels

# GraphRAG chain: NL → Cypher → Answer
chain = GraphCypherQAChain.from_llm(
    llm=llm,
    graph=graph,
    verbose=True,
    return_intermediate_steps=True
)

# Cross-domain queries the KG can answer:
questions = [
    "Which suppliers have had disruptions AND "
    "delivered ingredients to restaurants with violations?",

    "Which rooms in the hotel had both energy anomalies "
    "and maintenance work orders this month?",

    "Find customers who had support tickets AND "
```

```

"made purchases over 10,000 THB in the same week.",
]

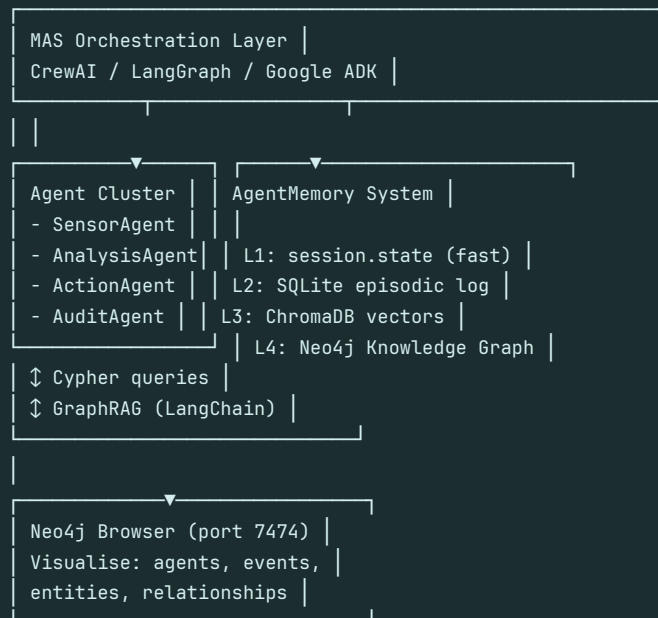
for q in questions:
    result = chain.invoke({"query": q})
    print(f"Q: {q}")
    print(f"A: {result['result']}")
    print(f"Cypher: "
          f"{result['intermediate_steps'][0]['query']}")
    print()

```

Key Insight: GraphRAG is the key to making your MAS explainable. When an agent makes a decision, it can log its reasoning as graph paths. Auditors can then ask "Why did the agent do X?" and get a Cypher-traversable explanation — a critical requirement for AI governance in regulated industries.

Final Architecture Diagram

■ Full system architecture



Appendix: Cypher Cheat Sheet & References

Quick reference for Neo4j knowledge graph queries

Essential Cypher Patterns

Create node	<code>CREATE (n:Label {prop: \$val})</code>
Merge (upsert)	<code>MERGE (n:Label {id: \$id}) SET n.prop = \$val</code>
Create relationship	<code>MATCH (a), (b) WHERE ... MERGE (a)-[:REL]->(b)</code>
Basic match	<code>MATCH (n:Label {prop: \$val}) RETURN n</code>
Pattern match	<code>MATCH (a)-[:REL]->(b:Label) RETURN a, b</code>
Multi-hop path	<code>MATCH (a)-[:REL*1..3]->(b) RETURN a, b</code>
Shortest path	<code>MATCH p=shortestPath((a)-[*]-(b)) RETURN p</code>
Aggregate	<code>MATCH (n)-[:REL]->(m) RETURN n, count(m) AS ct</code>
Filter by date	<code>WHERE n.ts > datetime() - duration({days: 7})</code>
Collect to list	<code>RETURN collect(DISTINCT n.prop) AS items</code>
Delete node+rels	<code>MATCH (n {id: \$id}) DETACH DELETE n</code>
Index creation	<code>CREATE INDEX FOR (n:Label) ON (n.prop)</code>

Key Python Libraries

Library	Version	Purpose	Install
neo4j	5.x	Official Neo4j Python driver	<code>pip install neo4j</code>
langchain-community	0.2+	Neo4jGraph, GraphCypherQChain	<code>pip install langchain-community</code>
langchain-experimental	0.1+	LLMGraphTransformer	<code>pip install langchain-experimental</code>
crewai	0.x	Multi-agent orchestration	<code>pip install crewai</code>
langgraph	0.1+	Stateful agent workflows	<code>pip install langgraph</code>
google-adk	1.x	Google Agent Development Kit	<code>pip install google-adk</code>
chromadb	0.5+	Vector store (semantic memory L3)	<code>pip install chromadb</code>

Reference Links

Neo4j LLM Graph Builder	neo4j.com/labs/genai-ecosystem/llm-graph-builder/
Neo4j Python Driver Docs	neo4j.com/docs/python-manual/current/
LangChain Neo4j Integration	python.langchain.com/docs/integrations/graphs/neo4j_cypher/
Google ADK Docs	google.github.io/adk-docs/agents/multi-agents/
CrewAI Docs	docs.crewai.com

LangGraph Docs	langchain-ai.github.io/langgraph/
z.ai API (GLM models)	open.z.ai → Profile → API Keys
Course GitHub	github.com/AltotechTH/agentic-coding-fitness
Woolridge MAS Textbook	Developing-Multi-Agent-Systems-with-JADE.pdf (workspace)

■ What to Build Next

1. Deploy the LLM Graph Builder (docker-compose up) and feed your workshop scenario PDFs into it — watch entities auto-appear in Neo4j Browser. 2. Connect your CrewAI/ADK agents to the shared Neo4j instance and run all 10 labs against the same knowledge graph — agents from different scenarios will start sharing and linking knowledge. 3. Add ChromaDB for semantic memory (L3) alongside Neo4j — use vectors for "find similar" and Cypher for "find related."